

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 6
Lists

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

- List Specifications
- List Implementations
  - Class Templates
  - Simply Linked Lists
  - Linked List Implementation
  - Doubly Linked
  - Orthogonal List
- List Examples

List Definition

- A **list** of elements of type T is a finite sequence of elements of T together with the following operations:

1. **Construct** the list, leaving it empty.
2. Determine whether the list is **empty** or not.
3. Determine whether the list is **full** or not.
4. Find the **size** of the list.
5. **Clear** the list to make it empty.
6. **Insert** an entry at a specified position of the list.
7. **Remove** an entry from a specified position in the list.
8. **Retrieve** the entry from a specified position in the list.
9. **Replace** the entry at a specified position in the list.
10.

Method Specifications

```
List::List();
```

Post: The List has been created and is initialized to be empty.

```
void List::clear();
```

Post: All List entries have been removed; the List is empty.

```
bool List::empty() const;
```

Post: The function returns **true** or **false** according to whether the List is empty or not.

```
bool List::full() const;
```

Post: The function returns **true** or **false** according to whether the List is full or not.

Method Specifications

int List :: size( ) **const**;

Post: The function returns the number of entries in the List.

Error_code List :: insert(**int** position, **const** List_entry &x);

Post: If the List is not full and $0 \leq$ position $\leq n$, where n is the number of entries in the List, the function succeeds: Any entry formerly at position and all later entries have their position numbers increased by 1, and x is inserted at position in the List.
Else: The function fails with a diagnostic error code.

Error_code List :: remove(**int** position, List_entry &x);

Post: If $0 \leq$ position $< n$, where n is the number of entries in the List, the function succeeds: The entry at position is removed from the List, and all later entries have their position numbers decreased by 1. The parameter x records a copy of the entry formerly at position.
Else: The function fails with a diagnostic error code.

Method Specifications

Error_code List :: retrieve(**int** position, List_entry &x) **const**;

Post: If $0 \leq$ position $< n$, where n is the number of entries in the List, the function succeeds: The entry at position is copied to x; all List entries remain unchanged.
Else: The function fails with a diagnostic error code.

Error_code List :: replace(**int** position, **const** List_entry &x);

Post: If $0 \leq$ position $< n$, where n is the number of entries in the List, the function succeeds: The entry at position is replaced by x; all other entries remain unchanged.
Else: The function fails with a diagnostic error code.

void List :: traverse(**void** (*visit)(List_entry &));

Post: The action specified by function *visit has been performed on every entry of the List, beginning at position 0 and doing each in turn.

Position Number in a List

- To find an entry in a list, we use an integer that gives its *position* within the list.
- We shall number the positions in a list so that the first entry in the list has position 0, the second position 1, and so on.
- Locating an entry of a list by its position is like indexing an array, but there are important differences. If we insert an entry at a particular position, then the position numbers of all later entries increase by 1. If we remove an entry, then the positions of all following entries decrease by 1.
- The position number for a list is defined without regard to the implementation. For a contiguous list, implemented in an array, the position will indeed be the index of the entry within the array. But we will also use the position to find an entry within linked implementations of a list, where no indices or arrays are used at all.

Class Templates

- We shall implement a **template class** *List* that depends on one generic type parameter. A client can then use our template to declare several lists with different types of entries with declarations of the following form: List<**int**> first_list; List<**char**> second_list;

```
template <class List_entry>
class List {
public:
   // methods of the List ADT
   List( );
   int size( ) const;
   bool full( ) const;
   bool empty( ) const;
   void clear( );
   void traverse(void (*visit)(List_entry &));
   Error_code retrieve(int position, List_entry &x) const;
   Error_code replace(int position, const List_entry &x);
   Error_code remove(int position, List_entry &x);
   Error_code insert(int position, const List_entry &x);
protected:
   // data members for a contiguous list implementation
   int count;
   List_entry entry[max_list];
};
```

Contiguous Implementation

Performance of Methods

- Operations
  - insert: O(n)
  - remove: O(n)
  - List: O(1)
  - clear: O(1)
  - empty: O(1)
  - full: O(1)
  - size: O(1)
  - replace: O(1)
  - retrieve: O(1)
  - traverse: O(n)

Simply Linked List Implementation

- **Node declaration:**

```
template <class Node_entry>
struct Node {
   // data members
   Node_entry entry;
   Node<Node_entry> *next;
   // constructors
   Node( );
   Node(Node_entry,
 Node<Node_entry>    *link =
NULL);
};
```

- **List declaration:**

```
template <class List_entry>
class List {
public:
   // Specifications for the methods of the
list ADT go here.
   // The following methods replace compiler-
generated defaults.
   ~List( );
   List(const List<List_entry> &copy);
   void operator = (const List<List_entry>
 &copy);
protected:
   // Data members for the linked list
implementation now follow.
   int count;
   Node<List_entry> *head;
   // The following auxiliary function is used
to locate list positions
   Node<List_entry> *set_position(int
 position) const;
};
```

Finding a List Position

- Function set position takes an integer parameter position and returns a *pointer* to the corresponding node of the list.
- To construct set position, we start at the beginning of the List and traverse it until we reach the desired node.

```
template <class List_entry>
Node<List_entry> *List<List_entry> ::
set_position(int position) const
/* Pre: position is a valid position in the List;
0 <= position < count.
Post: Returns a pointer to the Node in position. */
{
   Node<List_entry> *q = head;
   for (int i = 0; i < position; i++) q = q -> next;
   return q;
}
```

Traditional Linked List Implementation

- **Node declaration:**

```
typedef List_node* List_ptr;
struct List_node {
   List_entry data;
   List_ptr next ;
};

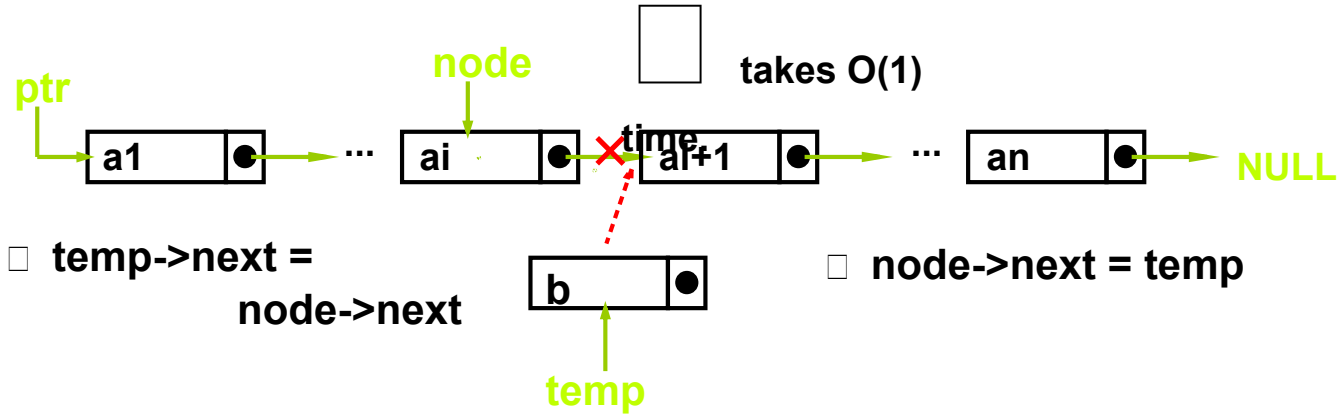
List_ptr ptr;
```

- **Usage:**

```
List_ptr N1, N2 ;
N1 = new List_node;
N2 = new List_node;
N1 -> data = "ZHAO";
N2 -> data = "QIAN";
N1 -> next = N2;
N2 -> next = NULL;
ptr = N1;
```

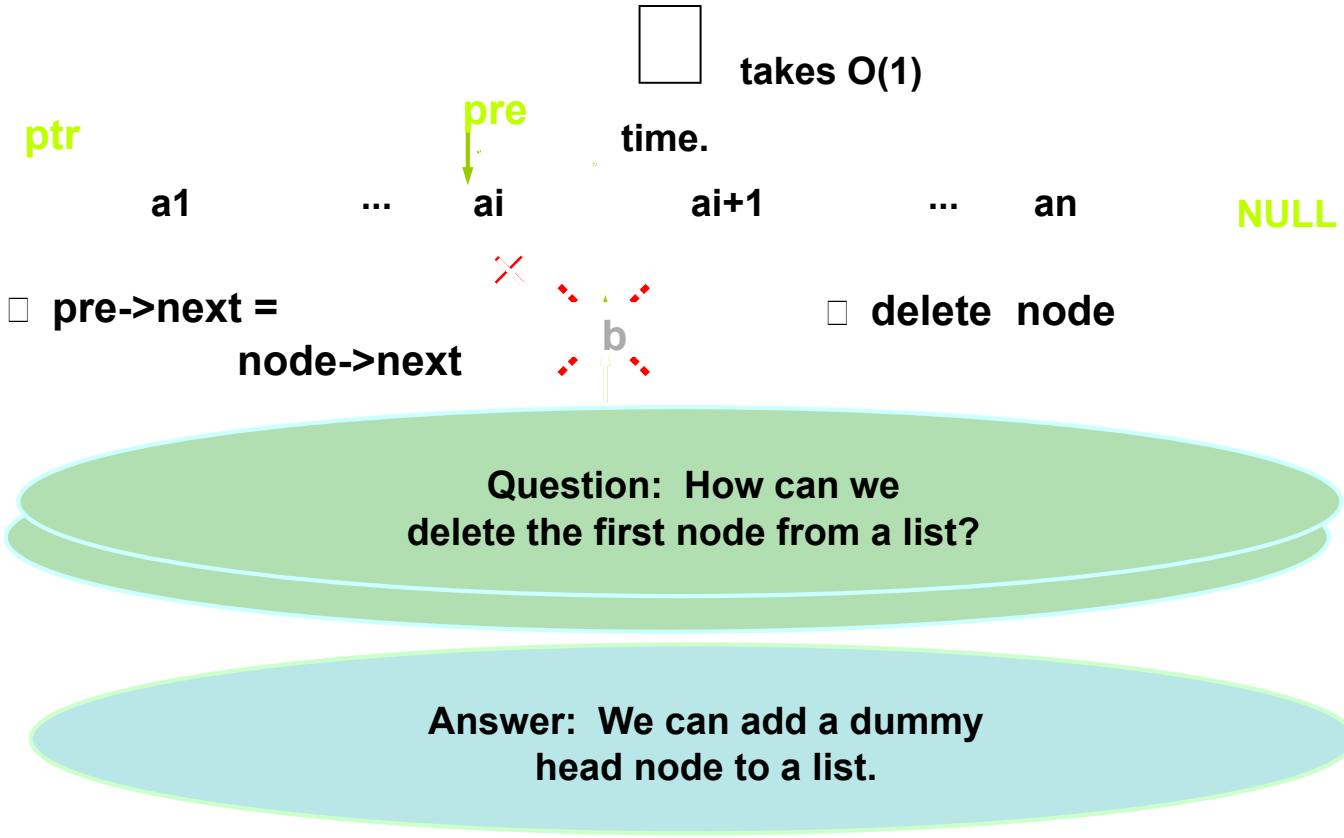


Node Insertion



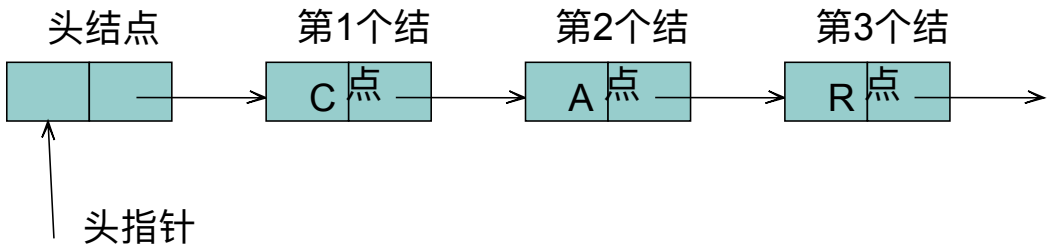
Question: How can we insert a new first item?

Node Deletion



Head Pointer and Head Node

- 链表的第一个结点的存储位置叫做头指针。
- 有时候为了更方便对链表进行操作，会在单链表的第一个结点前附设一个结点，称为头结点。头结点的数据域可以不存储任何信息，也可以存储线性表的长度等附加信息。头结点的指针域存储指向第一个结点的指针。

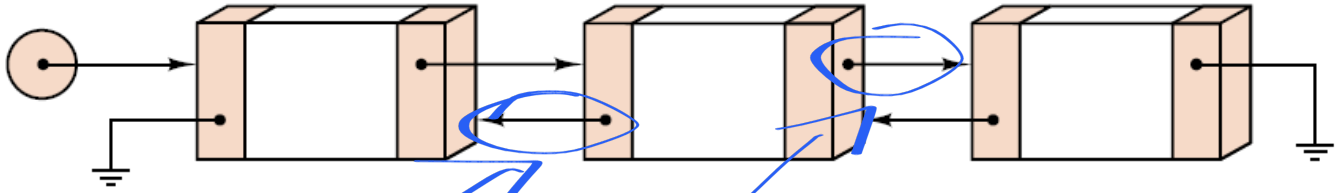


Head Pointer and Head Node

头指针	头结点
•头指针是指向第一个结点的指针，若链表有头结点，则是指向头结点的指针。 •头指针具有标识作用，所以常用头指针冠以链表的名字。 •无论链表是否为空，头指针	•头结点是为了操作的统一和方便而设立的，放在第一元素的结点之前，其数据域无意义。 •有了头结点，对在第一元素结点之前插入结点和删除第一结点，其操作与其他结点

Doubly Linked Lists

- Problem with simply linked list: if we frequently move both forward and backward through the list, the running time may be quite long.



```
template <class Node_entry>
struct Node {
   // data members
   Node_entry entry;
   Node<Node_entry> *next;
   Node<Node_entry> *back;
   // constructors
   Node( );
   Node(Node_entry, Node<Node_entry> *link_back = NULL,
   Node<Node_entry> *link_next = NULL);
};
```

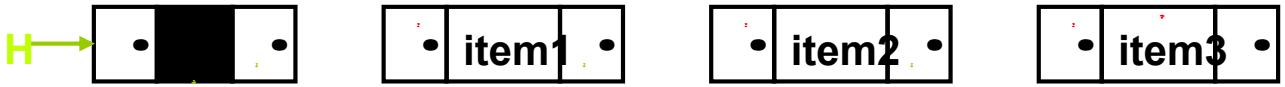
Doubly linked circular lists with head node

```
typedef  struct  node* node_ptr ;
struct  node  {
     node_ptr  llink;
     element    item;
     node_ptr  rlink;
} ;
```

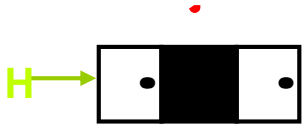


ptr = ptr->llink->rlink

= ptr->rlink->llink



An empty list :



STL Implementations

- **Contiguous implementation** is called vector.

```
#include <vector>
vector<int> v;
v.insert(v.begin(), 10);
```

- **Double linked lists** are called list.

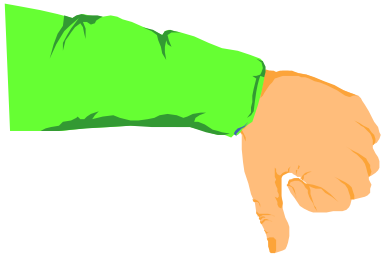
```
#include <list>
list<string> ls;
ls.insert(ls.begin(), “apple”);
```

Orthogonal List

- Suppose that we have **40,000** students and **2,500** courses. Print the students' name list for each course, and print the registered classes' list for each student.
- Representation 1:

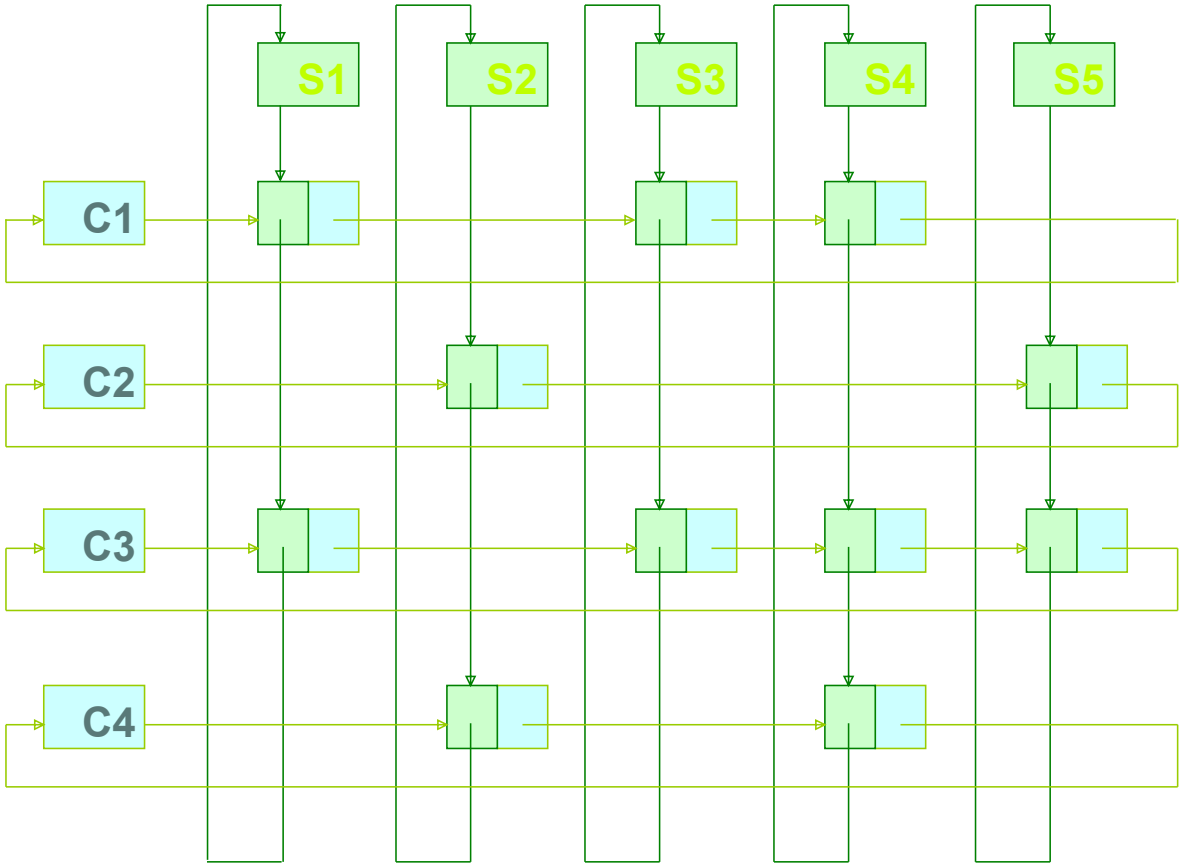
int Array[40000][2500];

$$\text{Array}[i][j] = \begin{cases} 1 & \text{if student } i \text{ is registered for course } j \\ 0 & \text{otherwise} \end{cases}$$



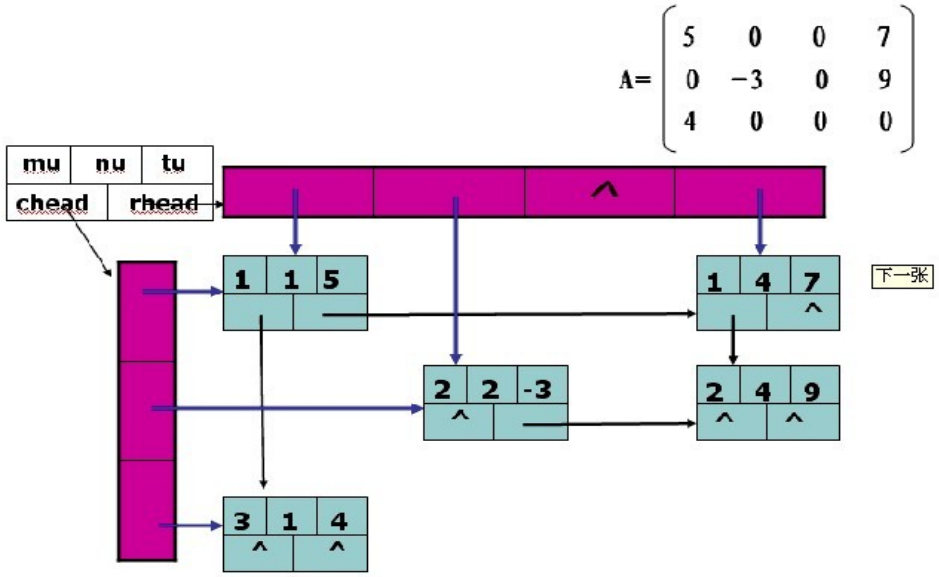
Orthogonal List

- Representation 2



*Orthogonal List

- 十字链表(Orthogonal List)是有向图的一种链式存储结构。十字链表可实现稀疏矩阵的存储。对于稀疏矩阵，三元组存储方式的实现很简单，每个元素有三个域分别是i, j, e。代表了该非零元的行号、列号以及值。那么在十字链表的存储方式下，将包括这三个域是肯定少不了的。而十字链表的形式可以理解成每一行是一个链表，而每一列又是一个链表。如图所示：



*Orthogonal List

```
struct OLNode {
    //非零元素的行和列下标
    int row, col;
    int value;
    //右边节点指针
    OLNode *right;
    //下方节点指针
    OLNode *down;
};
typedef OLNode *OLink;

//十字链表
struct CrossList {
  //十字行链表的头指针
  OLink *row_head;
  //十字列链表的头指针
  OLink *col_head;
  //稀疏矩阵的行数、列数和非零元素的个数
  int m, n, len;
};
```

Comparison of Implementations

Contiguous storage is generally preferable

- when the entries are individually very small;
- when the size of the list is known when the program is written;
- when few insertions or deletions need to be made except at the end of the list; and
- when random access is important.

Linked storage proves superior

- when the entries are large;
- when the size of the list is not known in advance; and
- when flexibility is needed in inserting, deleting, and re-arranging the entries.

Example: Swap

- **[例题]**设一系列正整数存放在一个数组中，试设计算法，将所有奇数存放在数组的前半部分，将所有的偶数存放在数组的后半部分。要求尽量少用临时存储单元并使时间最少。

Example: Swap

- 基本思想：

可以使用两个下标变量i和j分别指向数组的开头和末尾元素,并都向数组的中间进行扫描。

(1)若A[i]偶数、A[j]为奇数，则A[i]与A[j]进行交换,且i++; j--；

(2)若A[i]偶数、A[j]为偶数，则i保持不变，j--；

(3)若A[i]奇数、A[j]为奇数，则j保持不变，i++；

(4)若A[i]奇数、A[j]为偶数，则i++; j--；

(5)当i>=j时，算法结束。

- 算法：

Example: Swap

```
//用i和j分别指向数组的开、尾元素，并都想数组的中间进行搜索
void charge(int A[n])
{
  int i,j ch;
  i=0;j=n-1;
  while (i<j)
  {
     while(A[i]%2!=0)&&(i<j) i++;//A[i]不为偶数时，继续向右搜索
     while(A[j]%2==0)&&(i<j) j--; //A[j]不为奇数时，继续向左搜索
     if(i<j)                          // A[i]偶数、A[j]为奇数时，交换
     {   ch=A[i];
         A[i]=A[j];
         A[j]=ch; i++; j--;
     }
  }
}
```

Example: Delete Between

- **[例题]**已知线性表中的元素以递增有序排列，并以单链表作存储结构。试写一个高效的算法，删除表中所有值大于mink且小于maxk的元素(若表中存在这样的元素)同时释放被删除节点空间。

```
struct LinkNode {
    int data;
    LinkNode *next;
};
typedef LinkNode* LinkList;
```

Example: Delete Between

```
void Delete_Between(LinkList &head, int mink, int maxk)
{
    LinkList p = head;
    LinkList q = NULL;
    while (p && p->data < mink) {
        q = p;
        p = p->next;
    }
    if (p) {
        while (p && p->data <= maxk) {
            LinkList r = p;
            p = p->next;
            delete r;
        }
        if (q) q->next = p;
        else head = p;
    }
}
```

Think about it: Delete Duplicate

- 已知线性表中的元素以递增有序排列，并以单链表作存储结构。试写一个高效的算法，删除表中所有冗余的结点，即数据域相同的结点只保留一个。



- 链表结点定义如下：

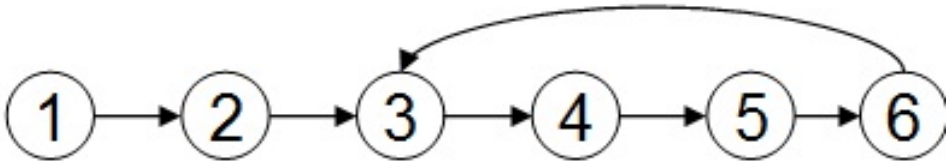
```
struct LinkNode {
    int data;
    LinkNode *next;
    LinkNode(int d, LinkNode *add_on = NULL) {
            data = d;
            next = add_on;
    }
};
typedef LinkNode* LinkList;
```

- 请实现函数：

```
void delete_duplicate(LinkList &head);
```

Think about it: Loop in Linked List

- 一个单向链表，怎么判断它是否存在环?



Homework

- 学习与分析以下容器的性能：静态数组, 动态数组, vector, list, deque
- 自行测试相关操作的运行时间
- 提交时间：9月27日，纸质版打印交回。
- 例如：

测试项目	静态数组	动态数组	vector	list	deque
初始化	0.00551	0.00461	0.207	1.30	0.352
前向遍历	0.00381	0.00549	0.0796	0.0756	0.0713
后向遍历	0.00422	0.00478	0.885	0.879	0.690
随机访问	0.000334	0.000342	0.00119	148	0.0115
后端插入	N/A	N/A	0.00192	0.0128	0.00260
后端移除	N/A	N/A	0.00131	0.0293	0.00194
前端插入	N/A	N/A	10.2	0.0128	0.00547
前端移除	N/A	N/A	10.3	0.0297	0.00135
中间插入	N/A	N/A	195	187	764
中间移除	N/A	N/A	195	209	753
反初始化	0.00139	0.00290	0.0000106	0.693	0.305

Thank you!

